Section A: Background

Multi-column layout is a common typographic technique used in books, newspapers, and technical documents. phppdf implements sequential column fill where each column is filled top-to-bottom before the next begins. When all columns on a page are exhausted, a new page is created and column flow continues from the top-left column of that new page.

Section B: Architecture

The column layout engine stores a ColumnLayout value object that tracks the number of columns, the gap width, and the current column index. The Page::cell() and Page::markdown() methods consult the active layout when deciding where to wrap and when to advance to the next column or page.

- The cursor x position is set to the left edge of the current column.
- The available width is the column width, not the full page width.
- An auto-break within a column advances to the next column, not a page.

Section C: Text Rendering

Markdown text is parsed into an AST and rendered node by node. Headings

are rendered in bold at scaled font sizes. Paragraphs wrap within the column width. Bullet lists indent items with a leading dash character. Inline emphasis and strong emphasis are rendered using the standard 14 font variants (italic and bold) without requiring custom font embedding.

Section D: Page Continuation

When markdown overflows the last column of a page, phppdf automatically adds a new page. The new page inherits the same document margins and the column layout resets to column zero at the top margin. This allows very long articles to span an arbitrary number of pages without any manual page management from the caller.

Section E: Summary

The multi-column feature is designed to be ergonomic. A single call to columns() with a render callback is all that is needed. The callback receives the current Page and calls the usual cell() or markdown() API. No column-awareness is required in the callback itself; the engine handles all the geometry automatically behind the scenes.